

Contents

1	Introduction	4
1.1	Purpose of the Document	4
1.2	Test Overview	4
2	Applicable and Reference Documents	5
3	Terms, Definitions and Abbreviated Items	6
3.1	Acronyms	6
3.2	Glossary	7
4	Test Specification	11
4.1	Unit and Integration Tests	11
4.2	Hardware-Software Tests	12
4.3	Acceptance Tests	13
5	Detailed Test Definition	14
5.1	Hardware-Software Test Procedures	14
5.2	Hardware-Software Test Procedures	15
5.2.1	Xentium Processing Network	15
6	Test Plan Requirements to Specification Traceability	17

List of Specified Tests

D-UIT-0001	11
D-UIT-0002	11
D-UIT-0003	11
D-UIT-0004	11
D-UIT-0005	11
D-UIT-0006	12
D-HST-0001	12
D-HST-0002	12
D-HST-0003	12
D-HST-0004	12
D-SCC-0001	13
D-ACT-0001	13
D-ACT-0002	13

Revision History

Revision	Date	Author(s)	Description
0.1	01.05.2017	AL	Initial version
1.0	01.06.2017	FK	Document approved
1.1	31.08.2026	AL	adopt FlightOS name

1. Introduction

1.1 Purpose of the Document

The test plan for the the FlightOS operating system is defined in [\[1\]](#).

This test plan includes tests where the integrated operating system is subjected to tests in its operational environment.

This document specifies the acceptance tests for FlightOS.

A test is specified by defining the:

- pre-conditions for the test
- hardware baseline
- system configuration
- checks to be performed
- pass/fail criteria

1.2 Test Overview

A list of tests specified in this document is available in chapter: [List of Specified Tests](#)

2. Applicable and Reference Documents

- [1] *FlightOS Test Plan*. 2026.
- [2] *FlightOS User Manual*. 2017.

3. Terms, Definitions and Abbreviated Items

3.1 Acronyms

ADC	Analog to Digital Converter
API	Application Programming Interface
BSP	Board Support Package
CPU	Central Processing Unit
DAC	Digital to Analog Converter
DMA	Direct Memory Access
DSP	Digital Signal Processor
ELF	Executable and Linkable Format
FIFO	First In - First Out
FPU	Floating Point Unit
GCC	GNU Compiler Collection
ILP	Instruction Level Parallelism
ISR	Interrupt Service Routine
MMU	Memory Management Unit
MPPB	Massively Parallel Processor Breadboarding system
NGAPP	Next Generation Astronomy Processing Platform
NoC	Network On Chip
POSIX	Portable Operating System Interface
PUS	Packet Utilisation Standard
RAM	Random-Access Memory
RISC	Reduced Instruction Set Computing
RMAP	Remote Memory Access Protocol
RR	Round Robin
SMP	Symmetric Multiprocessing
SoC	System On Chip
SSDP	Scalable Sensor Data Processor
TCM	Tightly-Coupled Memory
VLIW	Very Long Instruction Word

3.2 Glossary

Analog to Digital Converter (Analog to Digital Converter (ADC))

An Analog to Digital Converter is a system that converts an analog signal into a quantized digital signal. Its counterpart is the [Digital to Analog Converter \(DAC\)](#).

Application Programming Interface (Application Programming Interface (API))

The Application Programming Interface defines how a developer can write a program that requests services from an operating system or application. [APIs](#) are implemented by function calls composed of verbs and nouns, i.e. a function to execute on an object.

Board Support Package (Board Support Package (BSP))

A Board Support Package is the implementation of a specific interface defined by the abstract layer of an operating system that enables the latter to run on the particular hardware platform.

Central Processing Unit (Central Processing Unit (CPU))

The Central Processing Unit is the electronic circuitry that interprets instructions of a computer program and performs control logic, arithmetic, and input/output operations specified by the instructions. It maintains high-level control of peripheral components, such as memory and other devices.

Digital Signal Processor (Digital Signal Processor (DSP))

A Digital Signal Processor is a specialised processor with its architecture targeting the operational needs of digital signal processing.

Digital to Analog Converter (DAC)

A Digital to Analog Converter is a system that converts a quantized digital signal into an analog signal. Its counterpart is the [ADC](#).

Direct Memory Access (Direct Memory Access (DMA))

Direct Memory Access is a feature of a computer system that allows hardware subsystems to access main system [Random-Access Memory \(Random-Access Memory \(RAM\)\)](#) directly, thereby bypassing the [Central Processing Unit \(CPU\)](#).

Executable and Linkable Format (Executable and Linkable Format (ELF))

The Executable and Linkable Format is a common standard file format for executables, object code, shared libraries, and core dumps.

First In - First Out (First In - First Out (FIFO))

In FIFO processing, the "head" element of a queue is processed first. Once complete, the element is removed and the next element in line becomes the new queue head.

Floating Point Unit (Floating Point Unit (FPU))

A co-processor unit that specialises in floating-point calculations.

GNU Compiler Collection (GNU Compiler Collection (GCC))

The GNU Compiler Collection is a compiler system produced by the GNU project. It is part of the GNU toolchain collection of programming tools.

Instruction Level Parallelism (Instruction Level Parallelism (ILP))

Instruction-level parallelism (ILP) is a measure of how many instructions in a computer program can be executed simultaneously by the CPU.

Interrupt Service Routine (Interrupt Service Routine (ISR))

An Interrupt Service Routine is a function that handles the actions needed to service an interrupt.

Massively Parallel Processor Breadboarding system (Massively Parallel Processor Breadboarding system (MPPB))

The Massively Parallel Processor Breadboarding system is a proof-of-concept design for a space-hardened, fault-tolerant multi-DSP system with various subsystems to build a powerful digital signal processing system with a high data throughput. Its distinguishing features are the Network On Chip (Network On Chip (NoC)) and the Xentium DSPs controlled by a LEON2 processor. It was developed under ESA contract 21986 by Recore Systems B.V.

Memory Management Unit (Memory Management Unit (MMU))

A Memory Management Unit performs address space translation between physical and virtual memory pages and protects unprivileged access to certain memory regions.

Network On Chip (NoC)

A Network On Chip is a communication system on an integrated circuit that applies (packet based) networking to on-chip communication. It offers improvements over more conventional bus interconnects and is more scalable and power efficient in complex System On Chip (System On Chip (SoC)) designs.

Next Generation Astronomy Processing Platform (Next Generation Astronomy Processing Platform (NGAPP))

Next Generation Astronomy Processing Platform was an evaluation of the MPPB performed in a joint effort of RUAG Space Austria and the Department of Astrophysics of the University of Vienna. The project was funded under ESA contract 40000107815/13/NL/EL/f.

Packet Utilisation Standard (Packet Utilisation Standard (PUS))

The Packet Utilisation Standard addresses the end-to-end transport of telemetry and telecommand data between user applications on the ground and applications onboard a satellite. See also ECSS-E-70-41A.

Portable Operating System Interface (Portable Operating System Interface (POSIX))

The Portable Operating System Interface is a family of standards specified by the IEEE Computer Society for maintaining compatibility between operating systems.

Random-Access Memory (RAM)

Random-Access Memory is a type of memory where each memory cell may be accessed directly via their memory addresses.

Reduced Instruction Set Computing (Reduced Instruction Set Computing (RISC))

RISC is a CPU design strategy that intends to improve performance by combining a simplified instruction set with a microprocessor architecture that is capable of executing an instruction in a smaller number of clock cycles.

Remote Memory Access Protocol (Remote Memory Access Protocol (RMAP))

The Remote Memory Access Protocol is a form of SpaceWire communication that transparently communicates writes to memory mapped regions between different hardware devices.

Round Robin (Round Robin (RR))

Round Robin is a scheduling algorithm where time slices are assigned in equal portions and in circular order. In the context of threads, priorities are usually only used to control re-scheduling order when a mutex is accessed by a thread.

Scalable Sensor Data Processor (Scalable Sensor Data Processor (SSDP))

The Scalable Sensor Data Processor (SSDP) is a next generation on-board data processing mixed-signal ASIC, envisaged to be used in future scientific payloads requiring high-performance on-board processing capabilities. It is built upon a heterogeneous multicore architecture, combining two Xentium DSP cores with a general-purpose LEON3-FT control processor in a Network On Chip (NoC).

SpaceWire

SpaceWire is a spacecraft communication network based in part on the IEEE 1355 standard of communications.

Symmetric Multiprocessing (Symmetric Multiprocessing (SMP))

Symmetric Multiprocessing denotes computer architectures, where two or more identical processors are connected to the same periphery and are controlled by the same operating system instance.

System On Chip (SoC)

A System On Chip is an integrated circuit that combines all components of a computer or other electronic system into a single chip.

Tightly-Coupled Memory (Tightly-Coupled Memory (TCM))

Tightly-Coupled Memory is the local data memory that is directly accessible by a Xentium's load/store unit. It can be viewed as a completely program-controlled data cache.

Very Long Instruction Word (Very Long Instruction Word (VLIW))

Very Long Instruction Word is a processor architecture design concept that exploits [Instruction Level Parallelism \(ILP\)](#). This approach allows higher performance at a smaller silicon footprint compared to serialised instruction processors, as no instruction re-ordering logic to exploit superscalar capabilities of the processor must be integrated on the chip, but requires either code to be tuned manually or a very sophisticated compiler to exploit the full potential of the processor.

Xentium

The Xentium is a high performance [Very Long Instruction Word \(VLIW\) DSP](#) core. It operates 10 parallel execution slots supporting 32/40 bit scalar and two 16-bit element vector operations.

4. Test Specification

4.1 Unit and Integration Tests

D-UIT-0001	Identifier	Function
	Test Implementation	For each software component or module, a corresponding unit or integration test program is implemented by means of an adapted version of the <i>kselftest</i> framework. A shared collection of mockup and stub functions is maintained in order to facilitate testing.
Purpose	R-UIT-0001, R-UIITE-0001	

D-UIT-0002	Identifier	Function
	Test Coverage	All implemented unit tests deliver 100% statement, decision and condition coverage. Any deviations are justified as part of the unit test implementation.
Purpose	R-SCC-0004	

D-UIT-0003	Identifier	Function
	Test Execution	The collection of unit and integration tests is executable from the specified source directory via the <i>make</i> command.
Purpose	R-UIT-0003	

D-UIT-0004	Identifier	Function
	Test Results	The test executables generate human readable text output summarising the number, types and outcome of each test performed.
Purpose	R-UIT-0004	

D-UIT-0005	Identifier	Function
	Automated Testing	The test executables report failed tests as shell exit codes, so the outcome can be detected by an external test program such as <i>git bisect</i> .
Purpose	R-UIT-0005	

D-UIT-0006	Identifier	Function
	Test Suite	All unit and integration tests, along with stub and mockup functions are stored in the FlightOS source tree directory <i>tools/testing/unittest/</i> .
Purpose	R-UIT-0002, R-UIE-0001	

4.2 Hardware-Software Tests

D-HST-0001	Identifier	Function
	Test Implementation	For each software component or module that needs verification in a hardware environment, a test program is implemented.
Purpose Comment	R-HST-0001 Since these tests typically involve more complex control via other tools (such as <i>grmon</i> or SpaceWire interfaces), they are implemented as shell scripts that set up the required configuration and executes the test on the target hardware.	

D-HST-0002	Identifier	Function
	Test Suite	All hardware-software tests are stored in the FlightOS source tree directory <i>tools/testing/hwtests/</i> .
Purpose	R-HST-0002	

D-HST-0003	Identifier	Function
	Test Results	The test executables generate human readable text output summarising the number, types and outcome of each test performed.
Purpose	R-HST-0003	

D-HST-0004	Identifier	Function
	Automated Testing	The test executables report failed tests as shell exit codes, so the outcome can be detected by an external test program such as <i>git bisect</i> .
Purpose	R-HST-0004	

D-SCC-0001	Identifier	Function
	Code Cover- age	Statement, decision and condition coverage is determined via <i>gcov</i> . A <i>make</i> target <i>coverage_test</i> is available in the <i>Makefile</i> of the unit test directory of the FlightOS source tree. Human-readable reports are placed in the respective subdirectories for each individual unit test.
Purpose	R-SCC-0001, R-SCC-0002, R-SCC-0003	

4.3 Acceptance Tests

D-ACT-0001	Identifier	Function
	Accep- tance Test Defi- nition	Any acceptance test procedures are defined in chapter Detailed Test Definition of this document.
Purpose	R-ACT-0001	

D-ACT-0002	Identifier	Function
	Accep- tance Test Use Cases	The configuration and operational environment of FlightOS is defined for each acceptance test of an identified use case.
Purpose	R-ACT-0002	

5. Detailed Test Definition

5.1 Hardware-Software Test Procedures

The following tables list the tests specified in this document. Note that in the present version of the document, this list is *incomplete*, as the specification of the [SSDP](#) has not yet been released at this time.

Procedural steps are presented in tabular form and numerical sequence where applicable. Column headings are *TYPE*, *SEQ*, *Description* and *VERIFY*.

Procedural step *types* are:

CMT	Comment
PRE	Pre-condition
PST	Post-condition
STP	Step

SEQ indicates the sequence number of a step.

Description details the purpose and procedure of a step.

VERIFY lists any requirements or specifications verified in a step (if applicable).

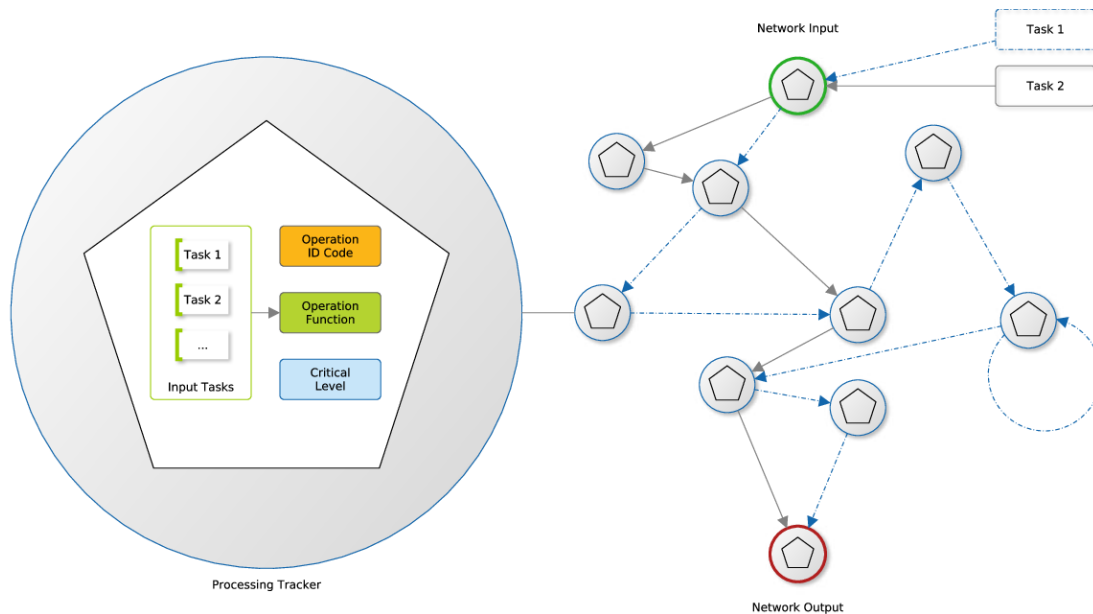


Figure 5.1: Tasks in the processing network propagate according to the operational code identifiers in their processing task list. For details, see [2].

5.2 Hardware-Software Test Procedures

5.2.1 Xentium Processing Network

This is an integrated test of the **Xentium** Processing Network (see Figure 5.1). It loads a set of **Xentium** program kernels in the Xentium driver of FlightOS, which are then used to apply processing operations on a series of input tasks to simulate a processing pipeline.

The Xentium kernels used in this test are:

- deglitch** applies a deglitching operation
- dummy** a simulation dummy that just passes on a task
- stack** collects multiple tasks and stacks their data into a single task
- rampfit** fits a linear function to a given number of samples

The test generates a total of 12 tasks (data sets) as input in groups of 4 tasks of 32, 64 and 64 samples of incremental value, containing a single glitch per data set. The stacking parameter is set to 4 and the ramp length is set to 32 samples.

The network then processes the tasks. Given 12 inputs, a stacking of 4 and a ramp length of 32, the network outputs 1 data point output for the first 4 task inputs of 32 samples and 2 data

points for inputs of 4x64 samples each, i.e. 5 data points in total. If this result is obtained, the test is considered a success.

ID	HWT-0001		
Purpose	Integrated System Test of Xentium Processing Network		
TYPE	SEQ	Description	VERIFY
CMT		This is a test to verify the function of the Xentium Processing Network implementation by generating a series of processing tasks and running them trough the processing network.	
PRE		a MPPB ver. 2.x hardware model	
PRE		<i>grmon</i> ver. 1.x	
PRE		Ensure connection of MPPB to test control host via serial cable on DEBUG UART to virtual device <i>/dev/ttyS0</i> on the host platform.	
PRE		A checkout of the FlightOS source directory.	
PRE		MPPB/Xentium toolchain installed and configured	
STP	1	Ensure that the MPPB is powered and reset.	
STP	2	Execute test script in the <i>tools/testing/hwtests/xen_proc_net</i> subdirectory of the FlightOS source tree.	
STP	3	Verify that the program image is built.	
STP	4	Verify that the program image is uploaded to the MPPB via <i>grmon</i>	
STP	5	Verify that the program is started on the MPPB .	
STP	6	Verify that the program reports SUCCESS on termination.	
CMT		Test completed.	

